

PBUI P01❖❖❖P04 Implementation Report

Results, composition hypotheses, and validation

PBUI research prototype

2026-08-05

Contents

1 PBUI P01-P04 Implementation Report	1
1.1 Status	1
1.2 Research objective	2
1.3 P01 result: identity is a sort-indexed equivalence relation	2
1.3.1 Main validated cases	3
1.3.2 Main unresolved question	3
1.4 P02 result: occurrence publication must be commit-phase and generation-scoped	3
1.4.1 Main validated cases	3
1.4.2 Main unresolved question	4
1.5 P03 result: unification makes selectors proof-relevant rather than Boolean	4
1.5.1 Main validated cases	4
1.5.2 Use of the unification chapter	4
1.6 P04 result: recursive UI semantics can use a finite least-fixed-point kernel	5
1.6.1 Main validated cases	5
1.6.2 Main unresolved question	5
1.7 Why P01 does not use the P03 unifier	5
1.8 Why P04 contains its own unifier	6
1.9 Composition hypothesis for the next pass	6
1.10 Validation summary	6
1.11 Recommended next experiments	7
1.12 Reproduction	7

1 PBUI P01-P04 Implementation Report

1.1 Status

Four research capsules have been implemented as independent TypeScript projects. Together they contain:

- 2,592 lines of TypeScript source;
- 44 executable tests;
- four runnable demonstrations;
- no runtime package dependencies;
- a reference implementation for each semantic subsystem;

- explicit limitations and proof obligations;
- a shared first-pass interoperability contract.

The command below builds and tests all four capsules:

```
npm test
```

All 44 tests pass under Node.js 22.16.0 and TypeScript 5.8.3 in the delivery environment.

1.2 Research objective

The implementation tests whether the first four PBUI subsystems can be separated without losing the information needed for later composition:

```
semantic identity
-> committed visual occurrence
-> inspectable selection
-> recursive derived applicability
```

The projects remain independent. This is a methodological choice rather than an unfinished integration. If P03 imported P01's internal registry or P04 imported P03's unifier during the first pass, their interfaces would reflect one implementation rather than independently investigated requirements.

1.3 P01 result: identity is a sort-indexed equivalence relation

P01 replaces ambient object/reference equality with an explicit semantic coordinate:

```
interface SubjectRef {
  sort: string;
  key: string;
}
```

The prototype supports three independent mechanisms:

1. **Key identity:** values of one sort map to stable keys.
2. **Representation identity:** a token or alternate visual value maps to the same sort/key.
3. **Alias identity:** explicit evidence-bearing edges generate an equivalence relation among keys of one sort.

This separation prevented several common conflation during implementation:

- a current snapshot is not the identity;
- a JavaScript object is not the identity;
- a display form is not the identity;
- an alias edge is not the entire equivalence class;
- a canonical representative is not the definition of sameness.

The durable alias graph retains provenance and supports removal. The current implementation recomputes connected components by graph traversal. That is deliberately slower but semantically clearer than persisting only a union-find root.

1.3.1 Main validated cases

- Project object and project-ID token identify one subject.
- Equal labels and equal object fields do not merge different keys.
- Equal key strings in different sorts remain different.
- Alias evidence is transitive and reversible by edge removal.
- Canonicalization is deterministic under link insertion order.
- Entity snapshots reject stale and same-revision conflicting data.

1.3.2 Main unresolved question

Should the production identity layer expose a single canonical reference or preserve a presented reference plus evidence of equivalence to a canonical reference? The implementation report recommends preserving both because audit, user explanation, and import migration need the original coordinate.

1.4 P02 result: occurrence publication must be commit-phase and generation-scoped

P02 models a visual occurrence as committed semantic state. Preparation is intentionally not visible:

```
prepare -> commit -> update* -> cleanup
```

Every committed occurrence ID has a generation. Update and remount produce a newer generation. Cleanup carries a lease and succeeds only for the currently active generation.

This solves a concrete race:

```
mount X@1  
mount replacement X@2  
late cleanup X@1
```

Without a generation check, the late cleanup deletes the replacement. With a lease, it becomes a no-op and an explicit lifecycle event.

The same generation is included in activation evidence. A delayed click cannot be accepted after an occurrence has been repurposed for another subject.

1.4.1 Main validated cases

- Prepared but abandoned renders leave no occurrence.
- Committing one preparation twice is idempotent.
- Old cleanup cannot remove a newer generation.
- Updates are atomic from the registry observer's perspective.
- Several occurrences can present one subject.
- Surface, visibility, reachability, and enabled status remain distinct.
- Activation evidence expires after update or unmount.
- Virtualization removes only the occurrence.

1.4.2 Main unresolved question

The pure commit protocol now needs at least two concrete React adapters for comparison: one based on layout effects and one based on an external-store registration service. Their semantic traces should be checked against the same P02 transition model before the production API is chosen.

1.5 P03 result: unification makes selectors proof-relevant rather than Boolean

P03 implements first-order terms and a syntactic unifier. A pattern such as:

```
field(?doc, ?name, "number")
```

returns bindings when matched against a ground field term. Those bindings can be consumed by the inspectable predicate language and retained in evidence.

This is materially stronger than a callback returning true:

Boolean callback:

```
accepted
```

unification-based selector:

```
accepted because pattern P unified with term T  
using substitution sigma  
and predicate evidence E held under sigma
```

The reference evaluator scans a sort. The compiled evaluator extracts simple equality indexes and then applies the complete semantics. Tests compare their candidate keys, preserving a slow semantic oracle for later optimizer work.

1.5.1 Main validated cases

- Constructor decomposition and variable elimination.
- Repeated-variable consistency.
- Occurs-check rejection.
- Substitution-composition law on examples.
- A more specific unifier factors through a computed general solution on the tested variables.
- Reference and indexed evaluators agree on fixture candidates.
- Dependency analysis supports serialization and index planning.
- Opaque JavaScript is visible as a guarantee downgrade.
- Evidence is invalidated by store revision changes.

1.5.2 Use of the unification chapter

The implementation follows the chapter's core syntactic distinctions:

- terms are generated from variables and constructors;
- substitutions extend through term structure;
- a useful solver constructs a general unifier, not only a yes/no answer;
- the occurs check prevents cyclic finite-term substitutions;

- syntactic and equational unification are distinct problems.

The report also adopts a caution from the chapter's category-theoretic reformulation: most-general unifiers and coequalizers have related factorization intuitions, but the uniqueness condition of a categorical coequalizer is stronger. The selector result is therefore called a unifier/substitution, not a quotient.

Primary source: <https://www.cs.bu.edu/fac/snyder/publications/UnifChapter.pdf>

1.6 P04 result: recursive UI semantics can use a finite least-fixed-point kernel

P04 implements a function-free, range-restricted rule language. The function-free restriction is important: without fresh term construction, a finite active constant domain yields only finitely many possible ground atoms at fixed arities.

The reference evaluator computes each stratum to quiescence. The positive agenda evaluator reacts to new facts and anchors rule-body occurrences to them. A persistent positive engine supports insertion-only incremental updates.

Provenance records the rule, substitution, positive premises, and checked negative atoms for each derivation. A certificate checker reconstructs the instantiated rule and recursively verifies premises.

1.6.1 Main validated cases

- Recursive transitive closure reaches the expected least fixed point.
- Agenda and naive evaluators agree on the fixture.
- Rule and base-fact order do not change the result.
- Re-evaluation from the completed fact set is idempotent.
- Stratified negation derives actions only in the completed lower-stratum absence case.
- Recursion through negation is rejected.
- Unsafe head/negative variables and function-generating terms are rejected.
- Unification enforces repeated-variable joins.
- Insertion-only incremental evaluation matches full recomputation on the fixture.
- Recursive derivations are explainable and checkable.

1.6.2 Main unresolved question

The generic rule engine and specialized algorithms overlap. Link connectedness, for example, can be derived by recursive rules, but a quotient compiler will use graph algorithms. The integration pass should treat P04 as the declarative specification and compare specialized implementations against its finite reference semantics.

1.7 Why P01 does not use the P03 unifier

The additional theoretical material might suggest treating subject identity as a unification problem. The implementation deliberately rejects that design.

Unification solves equations containing variables in a chosen term theory. P01 identity compares ground application coordinates under an application-defined key/alias theory.

Two structurally similar projects do not become one subject because a substitution exists between their descriptions.

The subsystems may interact later:

```
P03 pattern variable ?subject  
  binds to a P01 SubjectRef
```

but the substitution does not establish the validity of the P01 identity relation.

1.8 Why P04 contains its own unifier

The first-pass condition requires independent subsystem evaluation. P04 therefore contains a copy of the small first-order term implementation rather than importing P03.

This duplication creates a useful phase-two experiment:

1. compare both unifiers over a shared generated corpus;
2. define the minimum common interface;
3. determine whether selectors need features rules do not, or vice versa;
4. replace both only after differential conformance is established.

Premature sharing would hide those differences.

1.9 Composition hypothesis for the next pass

A composed selection operation can retain four evidence objects:

```
interface CompositeSelectionEvidence {  
  identity: IdentityEvidence;      // P01  
  occurrence: ActivationEvidence;  // P02  
  selector: SelectionEvidence;     // P03  
  derivation: RuleEvidence;        // P04  
}
```

A commit is accepted only if all four remain valid at their respective revisions. The revisions should remain a tuple, not be collapsed into one global counter.

The first integration constellation should implement this narrow flow:

1. register a P01 field subject;
2. commit two P02 occurrences presenting it;
3. export occurrences into a P03 selector store;
4. select one occurrence using a parameterized pattern;
5. export the selected candidate and capability facts to P04;
6. derive an inspect action;
7. mutate or unmount one input;
8. show which evidence component invalidates and why.

This is a better first integration target than building a complete React menu system.

1.10 Validation summary

Capsule	TypeScript source lines	Tests	Result
P01	476	9	pass
P02	405	11	pass
P03	843	13	pass
P04	868	11	pass
Total	2,592	44	pass

The checks establish executable behavior for the prototypes. They do not establish full metatheoretic soundness, completeness, or refinement to React. The project READMEs list proof obligations individually.

1.11 Recommended next experiments

1. **Differential unifier corpus:** run P03 and P04 against generated term-equation suites and compare substitutions modulo variable renaming.
2. **P01 quotient model in Lean:** prove equivalence laws and evidence-path soundness for finite alias graphs.
3. **P02 trace model:** express prepare/commit/update/cleanup in TLA+ and model stale cleanup and double activation.
4. **P03 compiler proof:** structurally prove that every extracted index is a sound candidate restriction and that full predicate evaluation preserves completeness.
5. **P04 evaluator proof:** mechanize fixed-point soundness for the positive fragment and validate agenda/reference equivalence.
6. **Evidence invalidation study:** compare coarse global revisions with dependency-specific version vectors.
7. **User explanation prototype:** render P03 and P04 evidence as concise “why selectable?” and “why available?” explanations.
8. **Specialized graph compiler:** compare P04 sameBinding closure with a P06-style quotient implementation.

1.12 Reproduction

```
cd pbui-p01-p04
npm test
npm run demo
```

No package installation is required when Node.js and TypeScript are already available.